

AKADEMIA GÓRNICZO – HUTNICZA IM. STANISŁAWA STASZICA W KRAKOWIE



Department of Computer Science, Electronics and Telecommunications

Electronics and Telecommunications

Project: System inwentaryzacji filamentów

Subject: Metodyki projektowania i modelowania systemów

Date: 15 czerwca 2026

Instructor: Bogusław Cyganek, prof. dr hab. inż

Students:

Aleksander Szczygielski

Marcin Szczepanowski

Spis treści

Lista oznaczeń	3
1 Wstęp	4
2 Wymagania systemowe	5
3 Funkcjonalność	6
4 Analiza problemu	7
4.1 Identyfikacja szpuli	7
4.2 Pomiar masy	7
4.3 Model danych i aktualizacja zużycia	7
5 Projekt techniczny	9
5.1 Architektura sprzętowa	9
5.2 Architektura oprogramowania (programowanie obiektowe)	10
5.2.1 Diagram klas	10
5.2.2 Architektura warstwowa	11
5.3 Schemat operacyjny	11
5.4 Zasada działania API	13
5.5 Konfiguracja	13
5.6 Model 3D obudowy	14
6 Infrastruktura i wdrożenie	16
6.1 Operacje sieciowe	16
6.2 Usługa Docker	16
6.3 Reverse proxy (Caddy)	17
6.4 Wdrożenie Ansible	17
7 Opis realizacji	18
8 Podręcznik użytkownika	21
9 Metodologia rozwoju i utrzymania systemu	22

Lista oznaczeń

ADC	Analog-to-Digital Converter (przetwornik analogowo-cyfrowy)
API	Application Programming Interface (interfejs programistyczny)
CAD	Computer-Aided Design (projektowanie wspomagane komputerowo)
CA	Certificate Authority (urząd certyfikacji)
DNS	Domain Name System
DTO	Data Transfer Object (obiekt transferu danych)
ESP32	Mikrokontroler firmy Espressif z wbudowanym WiFi/BT
HTTP	Hypertext Transfer Protocol
HTTPS	HTTP zabezpieczony warstwą TLS
JSON	JavaScript Object Notation
NVS	Non-Volatile Storage (partycja pamięci nieulotnej ESP32)
OOP	Object-Oriented Programming (programowanie obiektowe)
OTA	Over-the-Air (aktualizacja oprogramowania zdalnie)
REST	Representational State Transfer
RFID	Radio-Frequency Identification
SPI	Serial Peripheral Interface
SRP	Single Responsibility Principle
TLS	Transport Layer Security
UID	Unique Identifier (unikalny identyfikator tagu RFID)

Rozdział 1

Wstęp

Niniejszy dokument opisuje projekt oraz działanie inteligentnej wagi opartej na mikrokontrolerze ESP32, przeznaczonej do zarządzania szpulami filamentu do druku 3D. System współpracuje ze **Spoolman**, czyli otwartoźródłowym menedżerem inwentarza filamentu (<https://github.com/Donkie/Spoolman>), poprzez jego API typu REST, a do identyfikacji poszczególnych szpul wykorzystuje tagi RFID przyklejone do każdej z nich.

Głównym celem urządzenia jest automatyzacja śledzenia masy pozostałego filamentu. Gdy szpula zostaje położona na wadze, a jej tag RFID zbliżony do czytnika, urządzenie pobiera z bazy Spoolman dane o szpuli, mierzy jej masę całkowitą, oblicza masę samego filamentu i zapisuje zaktualizowaną wartość zużycia z powrotem do bazy. Dzięki temu stan magazynowy filamentu jest aktualizowany bez ręcznego ważenia i wpisywania danych.

Instancja Spoolman działa na serwerze w sieci laboratoryjnej koła naukowego (home-lab oparty na Proxmoxie), pod adresem `spoolman.aghrapid.top`. Kod układowy urządzenia jest wersjonowany w repozytorium <https://github.com/AGHRapidPro/spoolman-scale>. Raport opisuje zarówno warstwę sprzętową i programową samego urządzenia, jak i infrastrukturę serwerową, z którą się ono komunikuje.

Rozdział 2

Wymagania systemowe

1. Urządzenie po włączeniu łączy się ze skonfigurowaną siecią WiFi. Jeśli żadne dane logowania nie zostały zapisane lub połączenie się nie powiedzie, otwierany jest portal konfiguracyjny (captive portal) o nazwie **SpoolmanScale**, w którym użytkownik podaje dane sieci oraz adres serwera Spoolman. W przypadku złego wpisania danych lub chęci zmiany sieci, wymagane jest wymazanie pamięci flash microcontrolera w celu usunięcia danych zapisanych w partycji LittleFS.
2. Konfiguracja (adres URL serwera Spoolman oraz współczynnik kalibracji wagi) jest przechowywana w pamięci nieulotnej NVS i odczytywana przy każdym uruchomieniu.
3. Urządzenie identyfikuje szpule filamentu za pomocą czytnika RFID RC522, odczytując unikalny identyfikator (UID) tagu.
4. Aby powiązać UID z rekordem szpuli, urządzenie pobiera z API Spoolman pełną listę szpul i buduje w pamięci mapę odwzorowującą UID (pole `extra.rfid`) na identyfikator szpuli. Mapa jest tworzona raz i wykorzystywana ponownie przy kolejnych odczytach, a po nietrafieniu jednorazowo odświeżana. Szczegóły dopasowanej szpuli pobierane są dopiero osobnym żądaniem.
5. Urządzenie mierzy masę szpuli przy użyciu belki tensometrycznej sterowanej przez wzmacniacz HX711 oraz wyznacza stabilną wartość uśrednioną z wielu próbek.
6. Na podstawie pomiaru urządzenie oblicza masę filamentu (masa całkowita pomniejszona o masę pustej szpuli) i aktualizuje rekord w bazie Spoolman żądaniem **PATCH**.
7. Pętla główna działa w sposób ciągły, po każdym cyklu oczekując na kolejny tag RFID, bez konieczności ponownego uruchamiania urządzenia.
8. Urządzenie udostępnia zestaw komend diagnostycznych przez port szeregowy, w tym interaktywną kalibrację wzorcem 1 kg, a stan pracy i komunikaty błędów raportuje na konsoli szeregowej.

Rozdział 3

Funkcjonalność

- **Łączność WiFi z portalem konfiguracyjnym** oparta o bibliotekę WiFiManager, z automatycznym łączeniem i zapisem ustawień w NVS (czyli partycji littleFS).
- **Identyfikacja szpuli RFID** polegająca na odczycie UID tagu z czytnika RC522.
- **Wyszukiwanie szpuli w Spoolman**, które ze względu na brak w API zapytania po RFID pobiera listę wszystkich szpul, buduje w pamięci mapę UID do identyfikatora szpuli, a następnie pobiera szczegóły konkretnego rekordu.
- **Ważenie belką tensometryczną** przez HX711, z uśrednianiem próbek oraz algorytmem stabilizacji odczytu.
- **Automatyczna aktualizacja zużycia** filamentu w bazie poprzez żądanie PATCH aktualizujące pole `used_weight`.
- **Obsługa błędów** obejmująca logowanie braku szpuli lub serwera oraz jednorazowe odświeżenie pamięci podręcznej w razie nietrafienia.
- **Interfejs komend szeregowych** pozwalający odczytać masę, wykonać tarowanie, sprawdzić stan WiFi i serwera oraz uruchomić kalibrację.
- **Trwała pętla pracy** działająca bez przerwy pomiędzy kolejnymi pomiarami.

Rozdział 4

Analiza problemu

4.1 Identyfikacja szpuli

Każda fizyczna szpula filamentu ma przyklejony tag RFID. Czytnik RC522 odczytuje sprzętowy UID tagu i zamienia go na łańcuch szesnastkowy. Zasadniczym problemem jest powiązanie tego UID z rekordem szpuli w Spoolman, ponieważ API Spoolman nie udostępnia bezpośredniego zapytania w rodzaju „znajdź szpulę po RFID”.

Przyjęte rozwiązanie wykorzystuje pole `extra.rfid` rekordu szpuli, w którym zapisany jest UID tagu. Klient pobiera listę wszystkich szpul jednym żądaniem `GET /spool`, a następnie buduje w pamięci mapę `extra.rfid` do identyfikatora szpuli. Dalsza praca opiera się już na tej pamięci podręcznej, dzięki czemu kolejne odczyty tagów nie wymagają ponownego pobierania całej listy. W razie nietrafienia (na przykład gdy szpula została właśnie dodana) wykonywane jest jedno odświeżenie mapy i ponowna próba dopasowania.

4.2 Pomiar masy

Sygnał z belki tensometrycznej jest wzmacniany i przetwarzany przez wzmacniacz HX711. Zliczenia przetwornika są przeliczane na gramy przy użyciu współczynnika kalibracji ustawionego funkcją `set_scale`. W celu redukcji szumu pojedynczy odczyt jest średnią z N próbek:

$$m = \frac{1}{N} \sum_{i=1}^N r_i \quad (4.1)$$

gdzie r_i to kolejne odczyty przeliczone już na gramy. Aby uzyskać powtarzalny wynik, zastosowano algorytm stabilizacji oparty o okno ostatnich pięciu odczytów. Jeżeli różnica wartości maksymalnej i minimalnej spadnie poniżej zadanej tolerancji Δ_s , odczyt uznaje się za stabilny i zwraca środek przedziału:

$$m_{\text{stab}} = \frac{m_{\min} + m_{\max}}{2}, \quad m_{\max} - m_{\min} < \Delta_s \quad (4.2)$$

Domyślnie $\Delta_s = 1,5$ g, a algorytm wykonuje do trzydziestu prób; jeśli odczyt się nie ustabilizuje, zwracana jest ostatnia wartość wraz z ostrzeżeniem. Współczynnik kalibracji wyznacza się procedurą interaktywną: po wytarowaniu pustej wagi na platformie umieszcza się wzorzec 1000 g, a współczynnik obliczany jest jako iloraz średniego wskazania i masy wzorca, po czym zostaje zapisany w NVS.

4.3 Model danych i aktualizacja zużycia

Z rekordu szpuli pobierane są trzy istotne pola: `spool_weight` (masa pustej szpuli), `filament.weight` (masa filamentu dla pełnej szpuli) oraz `used_weight` (dotychczas zużyta masa filamentu). Masa

filamentu na zmierzonej szpuli oraz nowa wartość zużycia wyznaczone są następująco:

$$m_{\text{fil}} = \max(0, m_{\text{stab}} - m_{\text{szpula}}) \quad (4.3)$$

$$m_{\text{used}} = \max(0, m_{\text{fil,total}} - m_{\text{fil}}) \quad (4.4)$$

Tak wyliczone `used_weight` jest zapisywane do bazy. W bieżącej wersji aktualizacja następuje po każdym poprawnym pomiarze, bez progu różnicy; wartości ujemne są przycinane do zera, co zabezpiecza przed błędnym zapisem w razie niedoważenia.

Rozdział 5

Projekt techniczny

5.1 Architektura sprzętowa

Rdzeniem urządzenia jest moduł **Wemos LOLIN S2 Mini** oparty o układ ESP32-S2. Warto podkreślić, że ESP32-S2 jest mikrokontrolerem *jednordzeniowym*. Bieżąca, w dużej mierze blokująca architektura oprogramowania (oparta o opóźnienia i krótkie pętle aktywnego oczekiwania) mieści się na pojedynczym rdzeniu i okazała się wystarczająca dla obecnej funkcjonalności. Przy planowanej rozbudowie systemu, w szczególności przy jednoczesnym odpytywaniu czytnika RFID, obsłudze sieci i ważeniu w osobnych zadaniach FreeRTOS, zalecana będzie migracja na mocniejszy układ dwurdzeniowy, na przykład ESP32-S3.

Do pomiaru masy służy belka tensometryczna połączona ze wzmacniaczem **HX711**, komunikującym się z mikrokontrolerem dwuprzewodowo (przez I2C). Identyfikację szpul realizuje czytnik **RC522**. Czytnik jest podłączony przez programowy interfejs SPI. Przypisanie pinów zebrano w tabeli 5.1.

Tabela 5.1: Przypisanie pinów GPIO (plik `pins.h`)

Peryferium	Sygnał	GPIO
RC522 (RFID)	SS	13
RC522 (RFID)	RST	14
RC522 (RFID)	MISO	10
RC522 (RFID)	MOSI	11
RC522 (RFID)	SCK	12
HX711 (waga)	DT	16
HX711 (waga)	SCK	17

Tabela 5.2: Komponenty sprzętowe urządzenia

Komponent	Model	Interfejs
Mikrokontroler	Wemos LOLIN S2 Mini (ESP32-S2, jednordzeniowy)	UART
Wzmacniacz wagi	HX711 (24-bit ADC)	cyfrowy 2-przewodowy
Czytnik RFID	RC522	programowy SPI
Czujnik masy	belka tensometryczna	analogowy do HX711

5.2 Architektura oprogramowania (programowanie obiektowe)

Oprogramowanie urządzenia napisano w C++ budowanego narzędziem PlatformIO. Kod podzielono na pięć klas oraz prostą strukturę przenoszącą dane, a punktem spinającym całość jest plik `main.cpp`. Każda klasa odpowiada za jeden, jasno wydzielony obszar odpowiedzialności, zgodnie z zasadą pojedynczej odpowiedzialności (SRP):

- `Config` obsługuje trwałą konfigurację (adres URL Spoolman) w pamięci nieulotnej oraz uruchamia portal WiFiManager.
- `Scale` opakowuje przetwornik HX711, realizując tarowanie, kalibrację z zapisem w pamięci nieulotnej oraz stabilny odczyt masy.
- `RFIDReader` opakowuje układ czytnika nad programowym SPI i udostępnia metody `poll`, `halt` oraz `getUID`.
- `SpoolmanClient` jest klientem HTTP REST; utrzymuje pamięć podręczną mapowania UID na identyfikator szpuli i realizuje operacje pobrania oraz aktualizacji rekordu.
- `CommandHandler` to nieblokujący parser komend przychodzących z portu szeregowego, delegujący żądania do odpowiednich podsystemów.
- `SpoolInfo` jest strukturą przenoszącą dane jednej szpuli (identyfikator, masa pustej szpuli, masa filamentu, zużycie).

Zależności pomiędzy klasami widoczne są wprost w sposobie tworzenia obiektów w pliku `main.cpp`. Obiekty są komponowane raz, na poziomie modułu, a zależności wstrzykiwane przez referencje w konstruktorach:

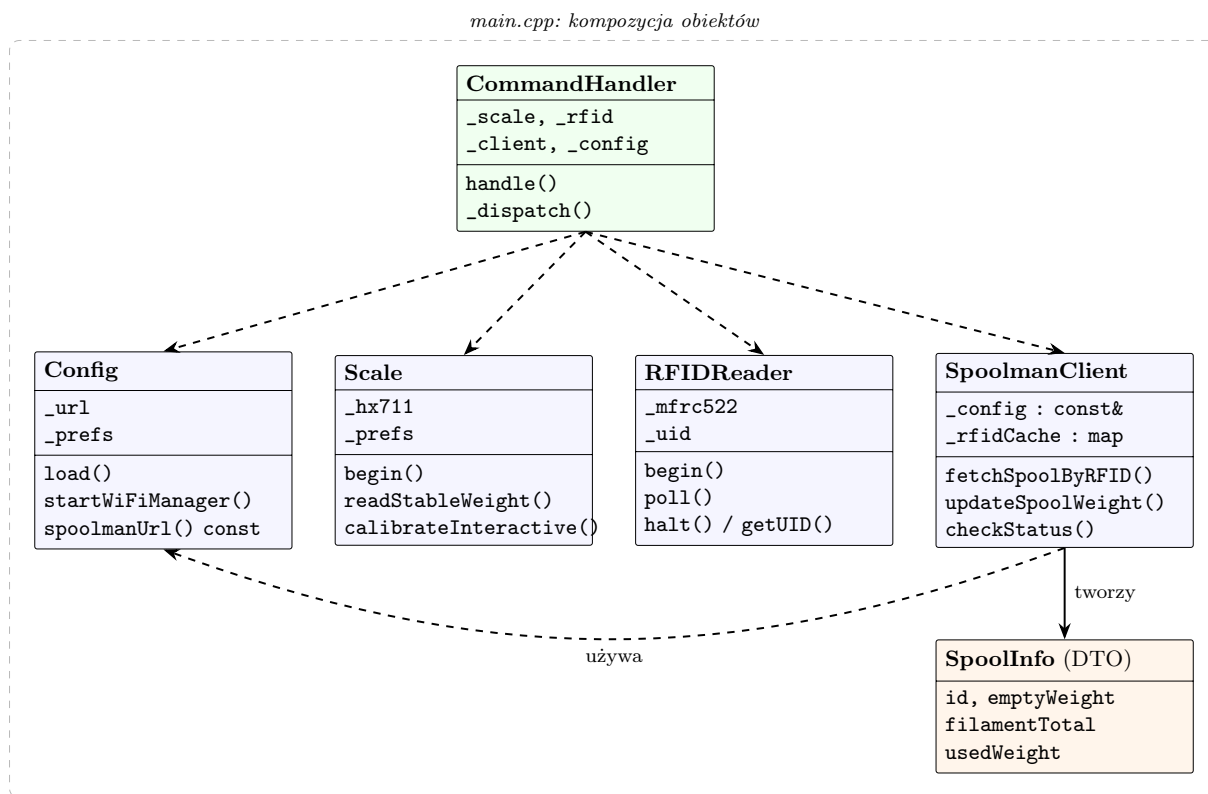
```
Config      config;  
Scale       scale(HX711_DT, HX711_SCK);  
RFIDReader  rfid(RFID_SS, RFID_RST, RFID_SCK, RFID_MISO, RFID_MOSI);  
SpoolmanClient spoolman(config); // const Config&  
CommandHandler commands(scale, rfid, spoolman, config); // referencje
```

Listing 5.1: Kompozycja obiektów i wstrzykiwanie zależności w `main.cpp`

W projekcie świadomie zastosowano kilka technik programowania obiektowego, które faktycznie występują w kodzie. **Enkapsulacja** polega na ukryciu detali implementacyjnych za prywatnymi polami i metodami pomocniczymi, takimi jak `_persist`, `_refreshCache`, `_baseUrl`, `_dispatch` czy `_saveCalibration`, przy zwięzłym interfejsie publicznym. **Kompozycja i wstrzykiwanie zależności** przez referencje (`const Config&`, `Scale&`, `RFIDReader&`, `SpoolmanClient&`) zapewniają luźne powiązanie modułów: na przykład `SpoolmanClient` przy każdym żądaniu odczytuje aktualny adres z `Config`, więc zmiana ustawień w portalu działa natychmiast, bez ponownej inicjalizacji klienta. **Poprawność stałych** (`const-correctness`) widać w metodach dostępowych `spoolmanUrl()` `const` oraz `getUID()` `const`, w referencjach `const` i w konstruktorze `explicit SpoolmanClient`. Klasy pełnią rolę **warstwy abstrakcji sprzętu**, ukrywając biblioteki producentów (HX711, MFRC522, WiFiManager, HTTPClient) za jednolitym interfejsem domeny problemu.

5.2.1 Diagram klas

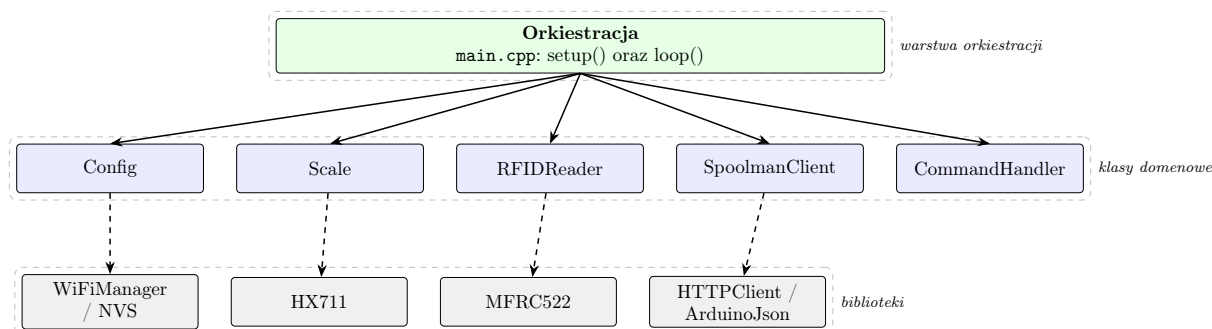
Rysunek 5.1 przedstawia diagram klas wraz z zależnościami. Przerywana ramka grupuje obiekty komponowane w `main.cpp`. Linia przerywana oznacza zależność przez wstrzykniętą referencję (relacja „używa”), a linia ciągła relację tworzenia obiektu danych `SpoolInfo`.



Rysunek 5.1: Diagram klas oprogramowania urządzenia (kompozycja i zależności)

5.2.2 Architektura warstwowa

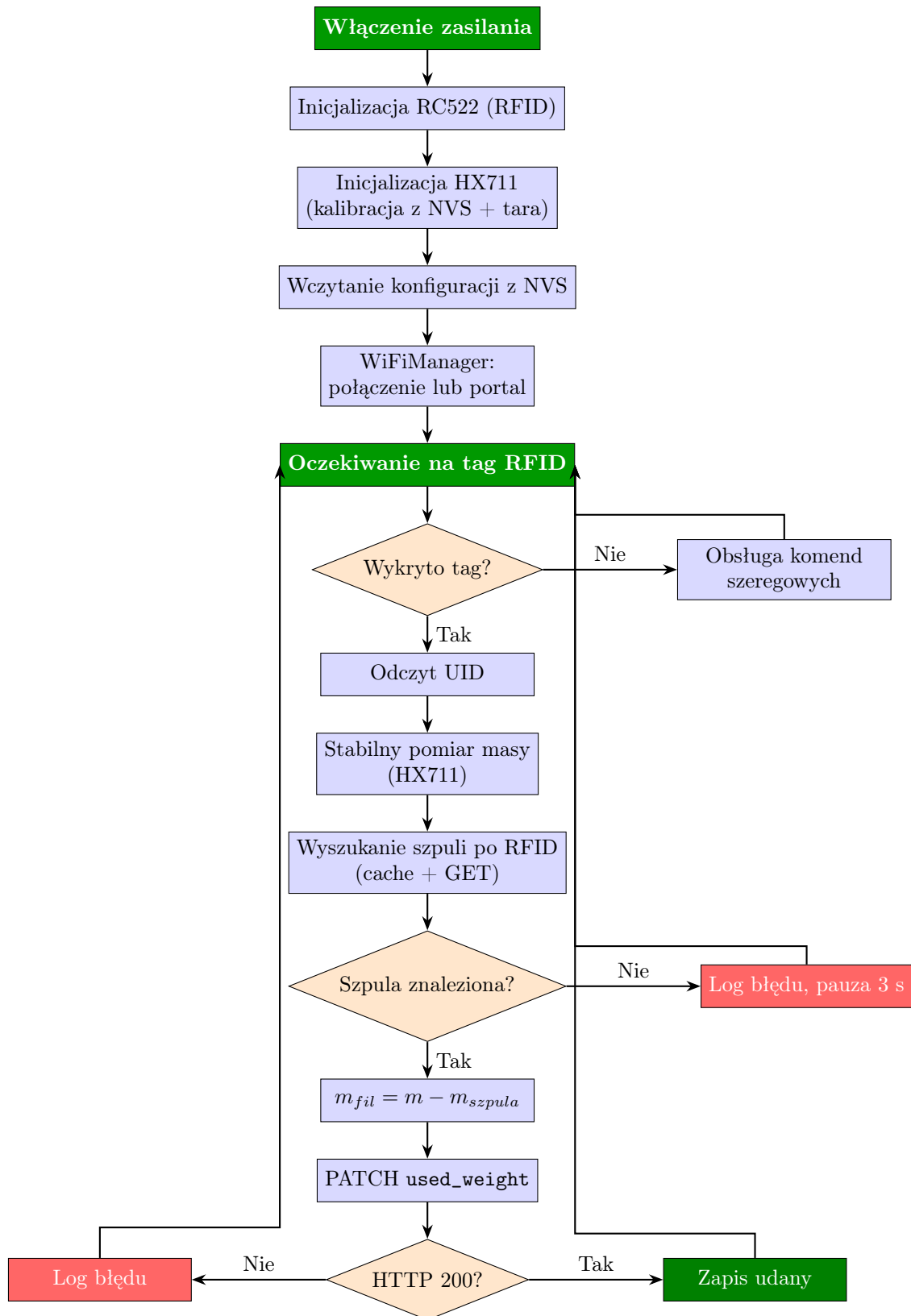
Z punktu widzenia organizacji modułów oprogramowanie układa się w trzy warstwy (rysunek 5.2): warstwę bibliotek sprzętowych i sieciowych, warstwę klas opakujących domenę problemu oraz warstwę orkiestracji w `main.cpp`. Klasy domenowe zależą wyłącznie od bibliotek poniżej, a warstwa orkiestracji spina je w przepływ pracy urządzenia.



Rysunek 5.2: Warstwowa architektura oprogramowania urządzenia

5.3 Schemat operacyjny

Rysunek 5.3 przedstawia pełny przebieg pracy urządzenia, od inicjalizacji peryferiów po pętlę pomiarową.



Rysunek 5.3: Schemat operacyjny urządzenia

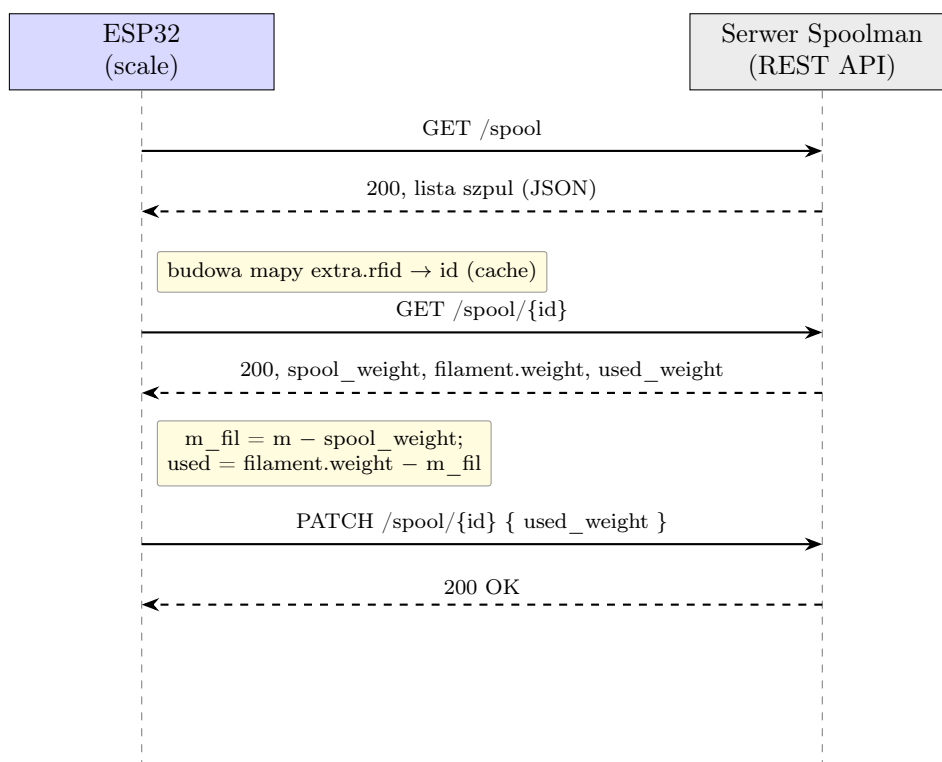
5.4 Zasada działania API

Komunikacja z serwerem opiera się o REST API Spoolman po protokole HTTP. Adres bazowy pochodzi z konfiguracji i jest normalizowany (uzupełniany o ukośnik końcowy) przez metodę `_baseUrl`. Klient wykorzystuje trzy operacje robocze oraz jedno żądanie diagnostyczne, zestawione w tabeli 5.3.

Tabela 5.3: Operacje API Spoolman używane przez urządzenie

Metoda + zasób	Cel	Przetwarzane pola
GET /spool	pobranie listy szpul i zbudowanie mapy cache	id, extra.rfid
GET /spool/{id}	pobranie szczegółów szpuli	spool_weight, filament.weight, used_weight
PATCH /spool/{id}	zapis nowego zużycia	treść JSON { used_weight }
GET / (bazowy)	sprawdzenie dostępności serwera	kod odpowiedzi HTTP

Ładunki JSON są przetwarzane biblioteką `ArduinoJson`, z dokumentami o rozmiarach dobranych do roli (większy bufor na listę szpul, mniejsze na pojedynczy rekord i na treść żądania `PATCH`). Diagram sekwencji na rysunku 5.4 ilustruje pełny cykl od odczytu tagu po zapis zaktualizowanego zużycia. Kluczowy jest tu krok pośredni: wobec braku zapytania po RFID w API, urządzenie najpierw pobiera listę szpul i lokalnie odwzorowuje UID na identyfikator, a dopiero potem sięga po szczegóły i zapisuje wynik.



Rysunek 5.4: Diagram sekwencji wymiany z API Spoolman

5.5 Konfiguracja

Urządzenie nie przechowuje danych wrażliwych w kodzie źródłowym. Dane sieci WiFi oraz adres serwera Spoolman wprowadza się jednorazowo w portalu konfiguracyjnym `WiFiManager`, a adres

zapisywany jest w przestrzeni `spoolman` pamięci NVS. Współczynnik kalibracji wagi jest przechowywany w osobnej przestrzeni `scale`. Listing 5.2 pokazuje rejestrację dodatkowego parametru portalu oraz zapis ustawień przez funkcję zwrotną.

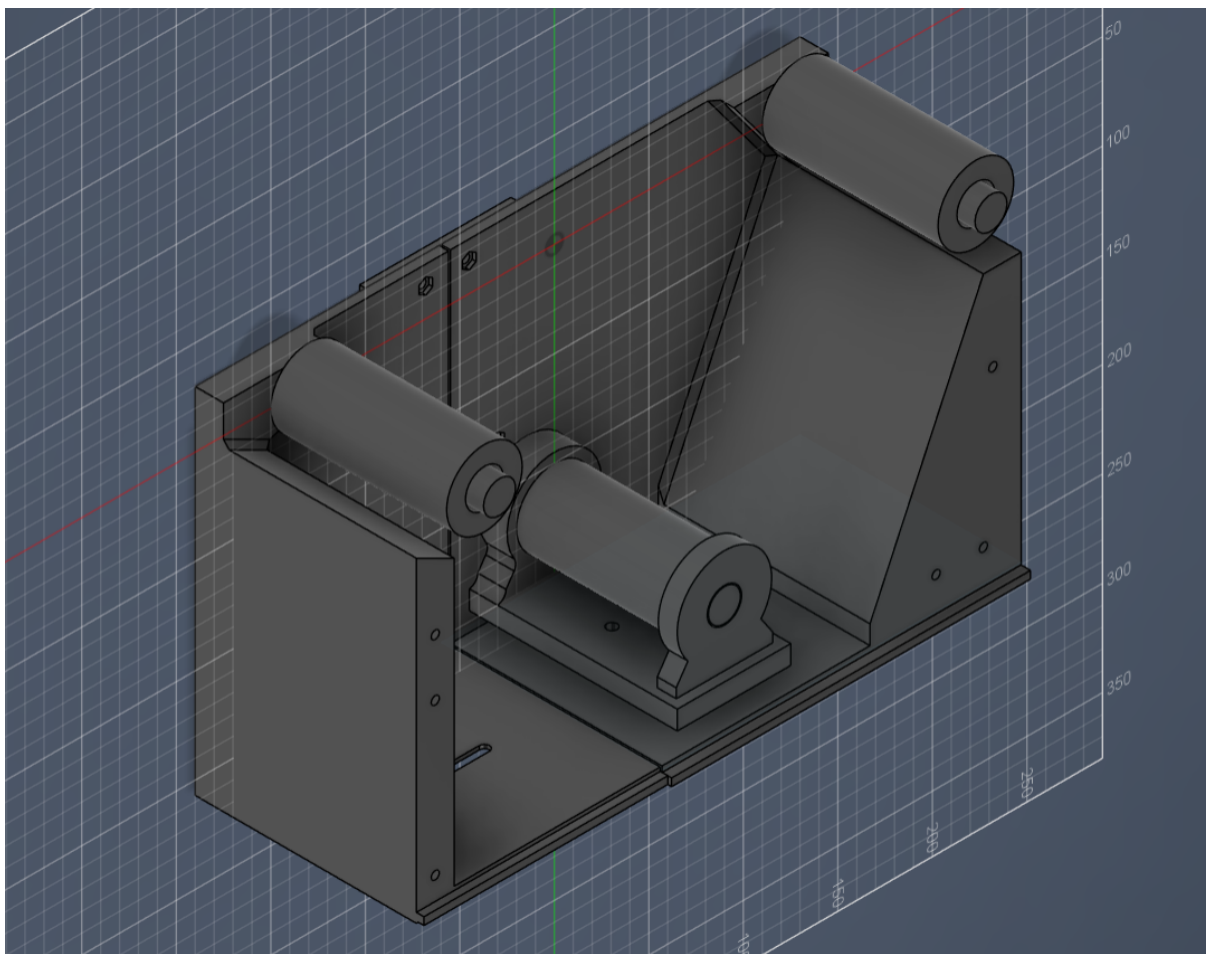
```
WiFiManagerParameter paramUrl("url",
    "Spoolman URL (e.g. http://192.168.1.100:7912/api/v1)",
    _url.c_str(), 100);
wm.addParameter(&paramUrl);

wm.setSaveParamsCallback([&]() {
    _url = paramUrl.getValue();
    _persist();                // zapis do NVS
});
wm.autoConnect("SpoolmanScale"); // SSID portalu
```

Listing 5.2: Parametr portalu i zapis konfiguracji (`Config.cpp`)

5.6 Model 3D obudowy

Obudowa urządzenia została zaprojektowana w środowisku CAD (model źródłowy `SpoolmanScale_test_model` w repozytorium) i wykonana techniką druku 3D. Mieści ona moduł ESP32, wzmacniacz HX711 oraz czytnik RC522, a na górze udostępnia platformę, na której opiera się belka tensometryczna i ważona szpula. Wizualizację modelu z programu CAD przedstawia rysunek ??, a zdjęcia wydrukowanej obudowy zamieszczono w rozdziale dotyczącym realizacji.

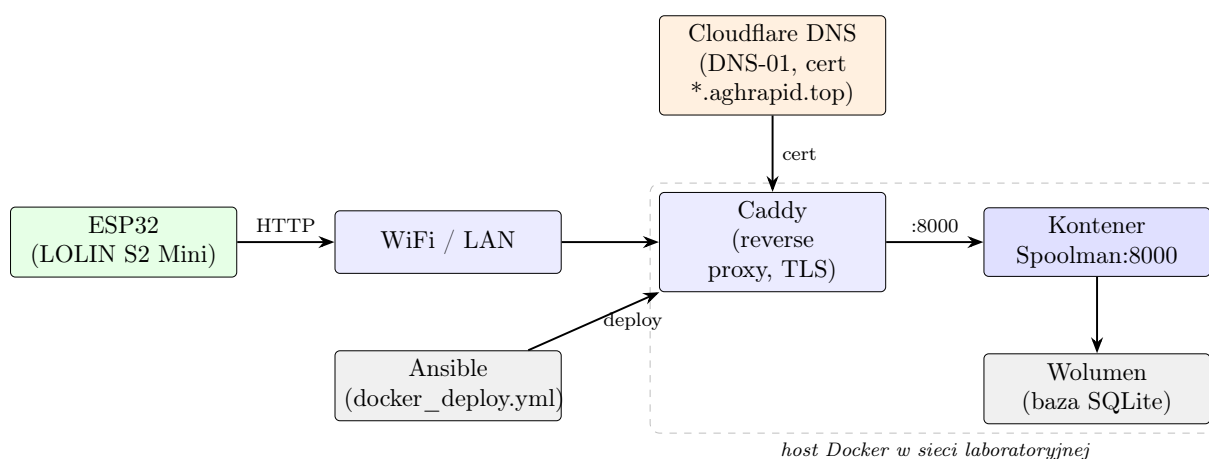


Rysunek 5.5: Model 3D obudowy (prototyp, ulegnie zmianie ze względu na brak odporności na wstrząsy)

Rozdział 6

Infrastruktura i wdrożenie

Instancja Spoolman jest częścią współdzielonego stosu Dockerowego laboratorium koła naukowego, zarządzanego przez Ansible i wystawionego na świat przez serwer pośredniczący Caddy. Rysunek 6.1 przedstawia ogólny obraz operacji sieciowych i wdrożenia: urządzenie ESP32 łączy się po WiFi i protokole HTTP z publicznym adresem, który serwer Caddy kieruje do kontenera Spoolman wewnątrz sieci Dockera. Stos jest wdrażany z repozytorium plików compose przez Ansible.



Rysunek 6.1: Architektura wdrożenia i operacji sieciowych

6.1 Operacje sieciowe

Urządzenie łączy się z serwerem przez publiczny adres po HTTP - tylko w sieci lokalnej.

6.2 Usługa Docker

Spoolman działa jako usługa `docker compose` obok innych narzędzi laboratorium (między innymi Vaultwarden, Wekan, LDAP, SuiteCRM) na hoście w sieci wewnętrznej. Definicję usługi przedstawia listing 6.1.

```
spoolman:
  image: ghcr.io/donkie/spoolman:latest
  container_name: spoolman
  user: root
  restart: unless-stopped
  volumes:
    - ./spoolman:/root/.local/share/spoolman
```



```

environment:
  - TZ=Europe/Warsaw
entrypoint: []
command:
  - uvicorn
  - spoolman.main:app
  - --host
  - 0.0.0.0
  - --port
  - "8000"

```

Listing 6.1: Definicja usługi Spoolman w Docker Compose

Dane trwałe (baza SQLite oraz konfiguracja) są przechowywane w katalogu `./spoolman/` na hoście i montowane do kontenera w `/root/.local/share/spoolman`.

6.3 Reverse proxy (Caddy)

Serwer Caddy z modułem Cloudflare obsługuje terminację TLS dla wszystkich usług laboratorium, używając certyfikatu typu wildcard dla domeny `*.aghrapid.top` wystawionego metodą DNS-01. Wirtualne hosty Spoolman zestawiono w tabeli 6.1; oba kierują ruch do `spoolman:8000` wewnątrz sieci Dockera.

Tabela 6.1: Wirtualne hosty Caddy dla Spoolman

Środowisko	Adres publiczny
Produkcja	<code>https://spoolman.aghrapid.top</code>
Przedprodukcja	<code>https://spoolman-preprod.aghrapid.top</code>

6.4 Wdrożenie Ansible

Stos jest wdrażany playbookiem Ansible (`docker_deploy.yml`) wymierzonym w dwie grupy hostów zdefiniowane w `inventory.yml` (tabela 6.2). Playbook synchronizuje pliki compose z katalogu `../docker-compose/<host>/` do `/opt/` na maszynie docelowej za pomocą `rsync`, wstrzykuje plik `secrets.env`, uwierzytelnia się w rejestrze Dockera, po czym uruchamia stos modułem `community.docker.docker_compose_v2`. Sekrety są usuwane z hosta docelowego bezpośrednio po wdrożeniu.

Tabela 6.2: Cele wdrożenia Ansible

Środowisko	Host Ansible	Wewnętrzny DNS
Produkcja	<code>docker</code>	<code>docker.rapid.internal</code>
Przedprodukcja	<code>docker-preprod</code>	<code>docker-preprod.rapid.internal</code>

Rozdział 7

Opis realizacji

Oprogramowanie zrealizowano na ESP32, budowanym i wgrywanym poprzez narzędzie PlatformIO (płytką `lolin_s2_mini`, prędkość monitora 115200). Wykorzystane biblioteki zewnętrzne zadeklarowano w pliku `platformio.ini`: `MFRC522` do obsługi RFID, `HX711` do wagi, `ArduinoJson` do przetwarzania JSON oraz `WiFiManager` do konfiguracji sieci. Kod podzielono na pliki źródłowe odpowiadające klasom opisanym w rozdziale technicznym, z osobnym plikiem `pins.h` zawierającym przypisanie pinów.

W trakcie realizacji powstało kilka rozwiązań wartych podkreślenia, które faktycznie znajdują się w kodzie.

Algorytm stabilnego odczytu masy. Zamiast pojedynczego pomiaru zastosowano okno przesuwne ostatnich pięciu uśrednionych odczytów. Pomiar uznawany jest za wiarygodny dopiero wtedy, gdy rozstęp w oknie spadnie poniżej zadanej tolerancji; algorytm wykonuje do trzydziestu prób i raportuje bieżącą zmienność na konsoli.

```
float variation = maxVal - minVal;
if (variation < maxVariation) {
    float stable = (minVal + maxVal) / 2.0f;
    return stable;
}
```

Listing 7.1: Sprawdzanie stabilności odczytu (`Scale.cpp`)

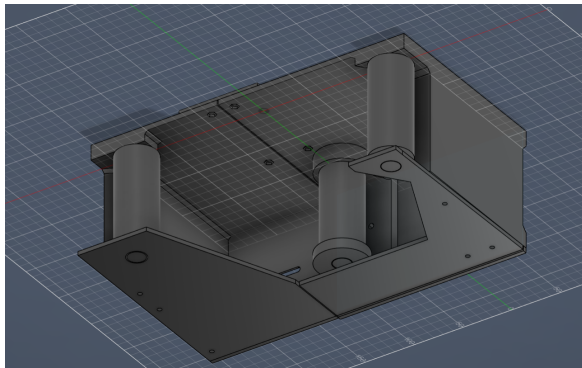
Pamięć podręczna RFID po stronie klienta. Ponieważ API Spoolman nie udostępnia wyszukiwania po RFID, klient pobiera listę szpul i buduje mapę `extra.rfid` do identyfikatora. Przy nietrafieniu wykonywane jest jedno odświeżenie i ponowna próba, co obsługuje przypadek świeżo dodanej szpuli bez stałego pobierania całej listy.

Trwałość w pamięci NVS. Zarówno współczynnik kalibracji wagi, jak i adres serwera Spoolman są zapisywane w pamięci nieulotnej przez bibliotekę `Preferences`, dzięki czemu ustawienia przetrwają restart i utratę zasilania.

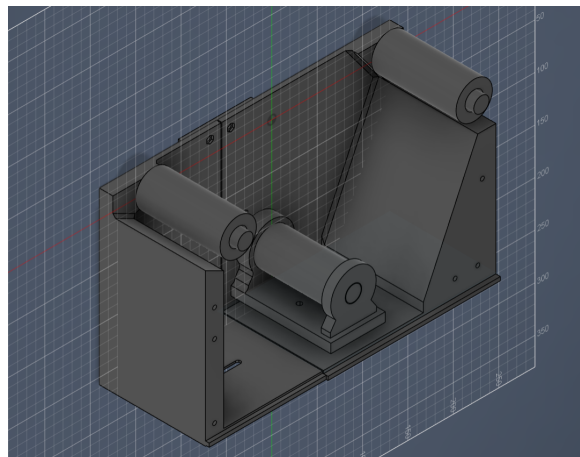
Portal konfiguracyjny WiFiManager. Pierwsza konfiguracja odbywa się bez przekompilowania kodu: urządzenie udostępnia sieć `SpoolmanScale` z formularzem WiFi oraz dodatkowym polem na adres Spoolman, zapisywanym przez funkcję zwrrotną.

Nieblokujący interfejs komend szeregowych. Klasa `CommandHandler` przetwarza co najwyżej jedną kompletną komendę na wywołanie i jest wpleciona w pętlę główną oraz w okna oczekiwania, dzięki czemu komendy diagnostyczne pozostają responsywne nawet podczas przerw po aktualizacji.

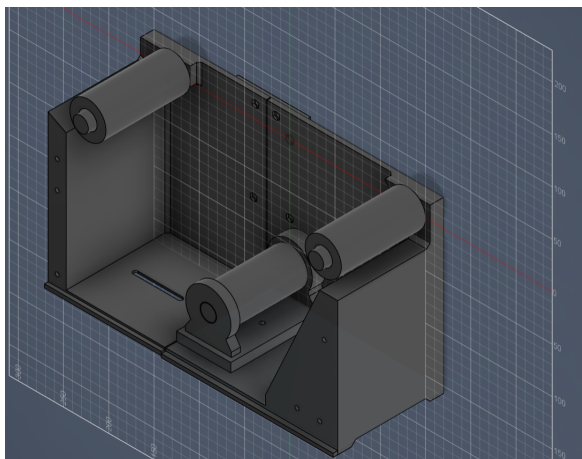
Zdjęcia zmontowanego prototypu urządzenia przedstawiono na rysunku 7.2. Miejsce na zdjęcia finalnej, wydrukowanej obudowy pozostawiono na rysunku ??.



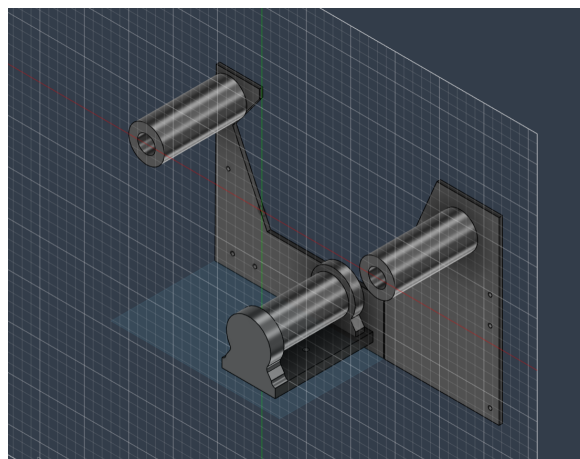
(a) Widok wewnątrz, można zobaczyć ułożenie rolek przytrzymujące szpule



(b) Widok ukazujący miejsce na belkę tensometryczną

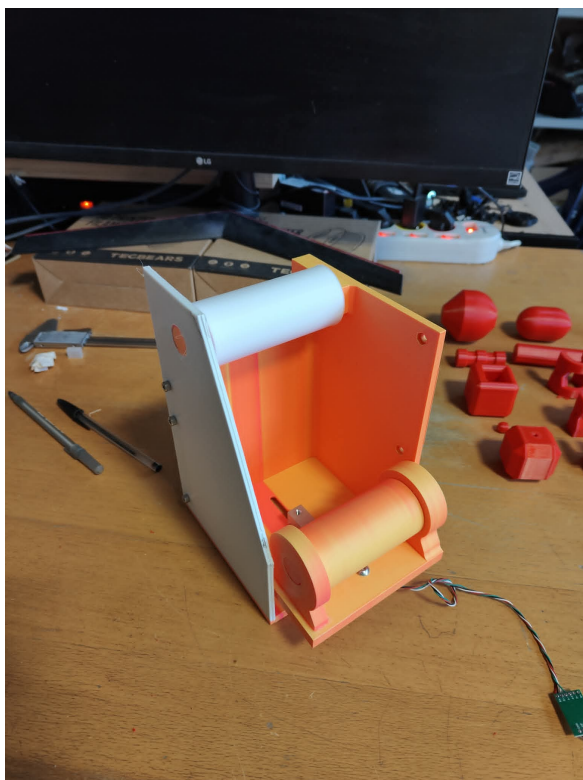


(c) Wskazanie zaczepu dolnego belki tensometrycznej

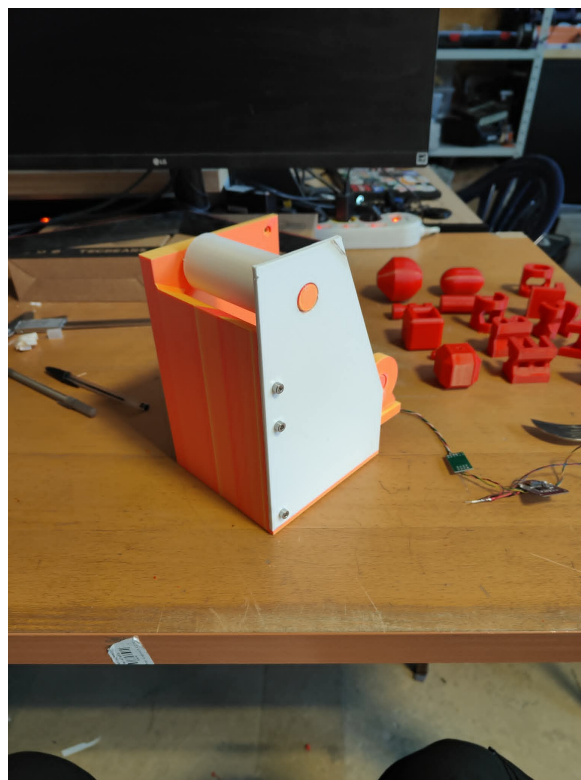


(d) Pozostałe mocowania (pomijając główne elementy konstrukcyjne)

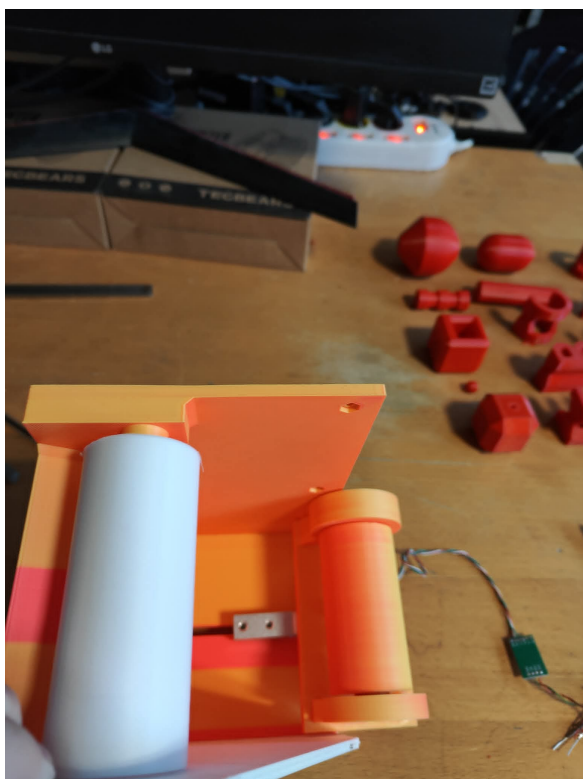
Rysunek 7.1: Prototyp urządzenia



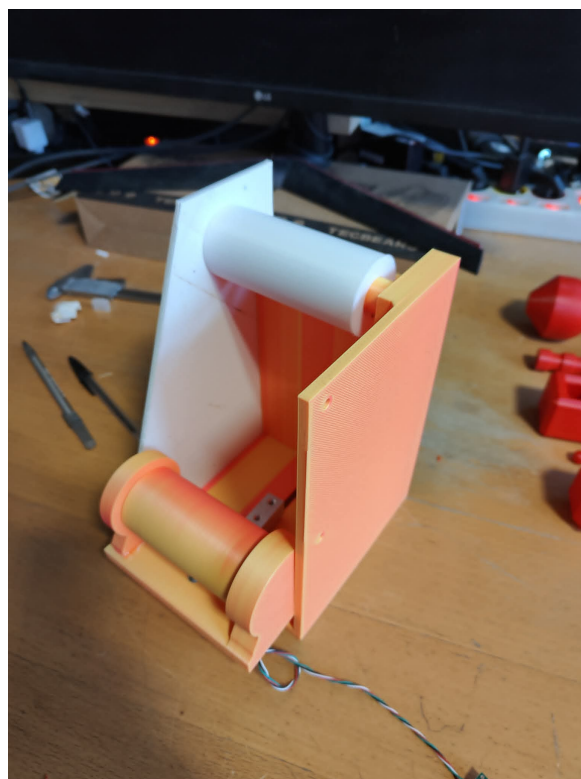
(a) Widok wewnątrz, można zobaczyć ułożenie rolki



(b) Widok ukazujący miejsce na belkę tensometryczną



(c) Widok boczny, od strony tensometru



(d) Widok boczny (kolejny)

Rysunek 7.2: Prototyp urządzenia

Rozdział 8

Podręcznik użytkownika

Korzystanie z urządzenia sprowadza się do kilku kroków. Przy pierwszym uruchomieniu, jeśli nie zapisano konfiguracji, urządzenie udostępnia sieć WiFi **SpoolmanScale**; należy się z nią połączyć i w otwartym formularzu podać dane własnej sieci oraz adres serwera Spoolman, a następnie zapisać ustawienia. Wagę kalibruje się jednorazowo komendą **ONE_KG_SCALE** wpisaną w monitor portu szeregowego: po zdjęciu obciążenia potwierdza się tarowanie, po czym umieszcza wzorzec 1000 g i ponownie potwierdza. UID tagu każdej szpuli należy przypisać do jej rekordu w Spoolman (pole **extra.rfid**).

Codzienne użycie polega na położeniu oznaczonej tagiem szpuli na wadze i zbliżeniu tagu do czytnika. Urządzenie odczyta UID, zmierzy masę, pobierze dane szpuli i zaktualizuje jej zużycie w bazie. Dostępne są też komendy diagnostyczne wpisywane przez port szeregowy: **WEIGHT** (bieżące wskazanie), **TARE** (tarowanie), **RFID** (ostatni UID), **SPL_STATUS** (dostępność serwera Spoolman) oraz **WIFI_STATUS** (stan sieci).

Rozdział 9

Metodologia rozwoju i utrzymania systemu

Dalszy rozwój systemu obejmuje zarówno warstwę sprzętową, jak i programową. Najważniejszym kierunkiem jest migracja na mocniejszy, dwurdzeniowy układ ESP (na przykład ESP32-S3). Obecny LOLIN S2 Mini jest jednordzeniowy, a architektura oprogramowania jest w dużej mierze blokująca; dwurdzeniowy mikrokontroler pozwoliłby rozdzielić odpytywanie RFID, obsługę sieci i ważenie na osobne zadania FreeRTOS, a tym samym poprawić responsywność i otworzyć drogę do aktualizacji OTA.

- ☐ Migracja na układ dwurdzeniowy (ESP32-S3) i podział pracy na zadania FreeRTOS.
- ☐ Docelowa strategia identyfikacji RFID: natywne wyszukiwanie po stronie serwera lub zapis identyfikatora wprost na tagu, aby uniknąć pobierania pełnej listy szpul.
- ☐ Próg aktualizacji (debouncing) ograniczający zbędne zapisy przy niewielkich wahaniach pomiaru.
- ☐ Wyświetlacz OLED z informacją zwrotną o masie i statusie.
- ☐ Aktualizacja oprogramowania w trybie OTA.
- ☐ Tryb offline z buforowaniem ostatnich danych szpuli w NVS na wypadek utraty WiFi.
- ☐ Weryfikacja certyfikatu serwera (przypięcie CA) w połączeniu HTTPS.

Bibliografia

- [1] Donkie, *Spoolman: Filament Spool Manager*, GitHub. <https://github.com/Donkie/Spoolman>
- [2] AGH Rapid Prototyping, *spoolman-scale: firmware repository*, GitHub. <https://github.com/AGHRapidPro/spoolman-scale>
- [3] Espressif Systems, *ESP32-S2 Technical Reference Manual*, 2023. <https://www.espressif.com/>
- [4] AVIA Semiconductor, *HX711: 24-Bit Analog-to-Digital Converter for Weigh Scales*, Datasheet, 2016.
- [5] NXP Semiconductors, *MFRC522: Standard performance MIFARE and NTAG Frontend*, Datasheet Rev. 3.9, 2016.
- [6] B. Blanchon, *ArduinoJson: C++ JSON library for embedded systems*. <https://arduinojson.org/>
- [7] tzapu, *WiFiManager: ESP WiFi configuration portal*. <https://github.com/tzapu/WiFiManager>
- [8] PlatformIO Labs, *PlatformIO: professional collaborative platform for embedded development*. <https://platformio.org/>